

Lambda 演算入门：Python 语法分析的视角

1. 引言

Lambda 演算 (Lambda Calculus) 是由数学家阿隆佐·邱奇 (Alonzo Church) 于 1930 年代提出的形式系统。它是最早的通用计算模型之一，与图灵机具有等价的计算能力。

核心思想：一切计算都可以通过函数定义和函数应用来表示。

Lambda 演算的极简语法：

```
<表达式> ::= <变量>           -- 变量引用
           | λ<变量>. <表达式> -- 函数抽象
           | <表达式> <表达式> -- 函数应用
```

2. Lambda 演算的 Python 实现

2.1 函数即一切

```
IDENTITY = lambda x: x    # λx.x
```

从 AST 视角看：

```
Module(Assign(targets=[Name('IDENTITY')],
                    value=Lambda(args=[x], body=Name(x))))
```

2.2 Church 编码：数与布尔

在 Lambda 演算中，可以用函数表示数据和逻辑：

```

# Church Numerals

ZERO  = lambda f: lambda x: x          #  $\lambda f.\lambda x.x$ 
ONE   = lambda f: lambda x: f(x)      #  $\lambda f.\lambda x.f\ x$ 
TWO   = lambda f: lambda x: f(f(x))   #  $\lambda f.\lambda x.f\ (f\ x)$ 


# Church Booleans

TRUE  = lambda t: lambda f: t          # 选择第一个参数
FALSE = lambda t: lambda f: f          # 选择第二个参数


# IF 条件: IF(p) (a) (b) = if p then a else b
IF     = lambda p: lambda a: lambda b: p(a) (b)


# Church Arithmetic

ADD    = lambda m: lambda n: lambda f: lambda x: m(f) (n(f) (x))


# 转换函数

to_int = lambda n: n(lambda k: k + 1) (0)

```

AST 视角：以 `TRUE` 为例

```
TRUE = lambda t: lambda f: t    #  $\lambda t.\lambda f.t$ 
```

从 AST 看，这是一个嵌套的 Lambda 表达式：

```

Lambda (
  args=[arg='t'],
  body=Lambda (
    args=[arg='f'],
    body=Name('t')    # 返回第一个参数 t
  )
)

```

字节码视角：`TRUE` 的执行

```

--- TRUE = lambda t: lambda f: t ---
1  LOAD_CONST (<code object <lambda>>)    # 加载内层 lambda
    MAKE_FUNCTION                          # 创建函数
    RETURN_VALUE

--- 内层 lambda (接收 t) ---
1  LOAD_FAST (t)                          # 加载闭包变量 t
    BUILD_TUPLE 1                          # 构建闭包元组
    LOAD_CONST (<code object <lambda>>)    # 加载最内层 lambda
    MAKE_FUNCTION                          # 创建带闭包的函数
    SET_FUNCTION_ATTRIBUTE (closure)       # 绑定闭包

--- 最内层 lambda (接收 f, 返回 t) ---
1  LOAD_DEREF (t)                         # 从闭包中取 t
    RETURN_VALUE

```

什么是闭包？ 闭包 (closure) 就是"函数 + 它捕获的外部变量环境"。在 `TRUE = lambda t: lambda f: t` 里，内层 `lambda f: t` 虽然形参只有 `f`，但它记住了外层的 `t`，所以之后仍能返回这个 `t`。

DEREF 是什么？ `DEREF` (如 `LOAD_DEREF`) 是字节码里"从闭包单元读取变量"的操作。也就是说，这个变量不在当前局部作用域 (`FAST`) 里，而是在外层作用域被捕获后存进闭包里，需要通过 `DEREF` 取出。

2.3 IF 条件的 AST 分析

```
IF = lambda p: lambda a: lambda b: p(a)(b)
```

AST 结构：

```

Lambda (args=[p], body=
  Lambda (args=[a], body=
    Lambda (args=[b], body=
      Call (
        # p(a) (b)
        func=Call (
          func=Name ('p'),
          args=[Name ('a')]
        ),
        args=[Name ('b')]
      )
    )
  )
)

```

字节码核心部分 (最内层) :

```

LOAD_DEREF (p)      # 加载谓词 p
CALL
LOAD_DEREF (a)      # 加载 then 分支 a
CALL                # p(a)
LOAD_FAST (b)       # 加载 else 分支 b
CALL                # p(a) (b)
RETURN_VALUE

```

这体现了 **Lambda 演算** 中 "控制流即函数应用" 的思想 : **IF** 不是语法结构 , 而是一个接受三个参数的函数。

2.4 ADD 的字节码深入分析

```
ADD = lambda m: lambda n: lambda f: lambda x: m(f) (n(f) (x))
```

下面我们追踪 **ADD (ONE) (TWO)** 的完整执行流程 :

第一步 : ADD 本身的字节码

```
ADD = lambda m: lambda n: lambda f: lambda x: m(f)(n(f)(x))
```

```
--- 外层 lambda (接收 m) ---  
LOAD_FAST (m)  
BUILD_TUPLE 1          # 将 m 加入闭包  
LOAD_CONST (<lambda>) # 加载下一层 lambda  
MAKE_FUNCTION  
SET_FUNCTION_ATTRIBUTE (closure)  
RETURN_VALUE          # 返回等待 n 的函数
```

第二步：ADD(ONE) 返回等待 n 的函数

```
--- 第二层 lambda (接收 n) ---  
COPY_FREE_VARS 1      # 复制 m 到闭包  
LOAD_FAST (m)  
LOAD_FAST (n)  
BUILD_TUPLE 2          # 将 m, n 都加入闭包  
LOAD_CONST (<lambda>) # 加载下一层 lambda  
MAKE_FUNCTION  
SET_FUNCTION_ATTRIBUTE (closure)  
RETURN_VALUE          # 返回等待 f 的函数
```

第三步：ADD(ONE)(TWO) 返回等待 f 的函数

```
--- 第三层 lambda (接收 f) ---  
COPY_FREE_VARS 2      # 复制 m, n 到闭包  
LOAD_FAST (f)  
LOAD_FAST (m)  
LOAD_FAST (n)  
BUILD_TUPLE 3          # 将 m, n, f 都加入闭包  
LOAD_CONST (<lambda>) # 加载最内层 lambda  
MAKE_FUNCTION  
SET_FUNCTION_ATTRIBUTE (closure)  
RETURN_VALUE          # 返回等待 x 的函数
```

第四步：最终执行 (当传入 f 和 x 时)

```
--- 最内层 lambda (接收 x, 执行计算) ---  
COPY_FREE_VARS 3      # 复制 m, n, f 到闭包  
  
# 计算 m(f)  
LOAD_DEREF (m)  
CALL, LOAD_DEREF (f), CALL  
  
# 计算 n(f)(x)  
LOAD_DEREF (n), CALL, LOAD_DEREF (f), CALL, LOAD_FAST (x), CALL  
  
# m(f)(n(f)(x))  -- 前面的结果作为函数调用后面的结果  
CALL  
RETURN_VALUE
```

性能分析：每次 `CALL` 都是函数调用开销，`ADD(ONE)(TWO)` 需要多次 `CALL` 才能完成。这解释了为什么 Lambda 递归比普通递归慢得多。

2.5 柯里化的本质

从上述字节码分析可以看出，柯里化 (Currying) 的代价：

1. **多层闭包：**每个参数都创建一层新的闭包，需要 `BUILD_TUPLE` 和 `SET_FUNCTION_ATTRIBUTE`
2. **变量捕获：**使用 `LOAD_DEREF` 从闭包中取值，比直接访问局部变量慢
3. **多次函数调用：**`m(f)(n(f)(x))` 需要 5 次 `CALL` 指令

不过，这种所有函数都是单参数的统一性，正是 Lambda 演算极简而优雅的来源。

3. Lambda 演算的性能思考

既然已经了解了它的基本运作方式，接下来就看一个更实际的问题：它在工程里到底好不好用？

从理论上讲，Lambda 演算非常优雅——用纯函数表示一切。但实际运行起来，这种“函数套函数”的方式效率如何？我们来做个简单的实验：用 Fibonacci 数列来对比普通递归和 Lambda 递归 (Church 数) 的性能差异。

3.1 Python 中的性能对比

```
# 普通递归
def fib_normal(n):
    if n <= 1:
        return n
    return fib_normal(n - 1) + fib_normal(n - 2)

# Lambda 递归 (Church 数)
def fib_lambda(n):
    if n <= 1:
        return to_church(n)
    return add(fib_lambda(n - 1))(fib_lambda(n - 2))
```

运行结果：

n	普通递归耗时	Lambda 递归耗时	慢
10	7μs	72μs	10x
12	13μs	146μs	11x
14	34μs	516μs	15x
16	90μs	1431μs	16x
18	185μs	3641μs	20x

结论：在 Python 中，Lambda 递归比普通递归慢 **10-20 倍**，且差距随 n 增大而扩大。

3.2 C++ 的表现

如果换成编译型语言 C++，情况会不会不一样？下面来看结果：

```
// C++ 普通递归
long long fib_normal(long long n) {
    if (n <= 1) return n;
    return fib_normal(n - 1) + fib_normal(n - 2);
}

// C++ Lambda 递归 (模仿 Church 数)
using Church = function<function<long long(long long)>(function<long long(long long)>)>>;
Church fib_lambda(long long n) {
    if (n <= 1) return to_church(n);
    return add(fib_lambda(n - 1), fib_lambda(n - 2));
}
```

C++ 运行结果：

n	C++ 普通递归	C++ Lambda 递归	慢
10	~0μs	105μs	>100x
12	1μs	157μs	157x
14	1μs	457μs	457x
16	2μs	1293μs	647x
18	5μs	3764μs	753x

3.3 四种方式汇总对比

将四种方式放在一起：

n	Python 普通	Python Lambda	C++ 普通	C++ Lambda
10	7μs	72μs	~0μs	105μs
14	34μs	516μs	1μs	457μs
18	185μs	3641μs	5μs	3764μs

关键发现：

1. **Lambda 递归确实慢**：无论 Python 还是 C++，Lambda 递归都比普通递归慢得多
2. **语言差异**：C++ 普通递归比 Python 快 10-40 倍，但 C++ Lambda 与 Python Lambda 速度相当
3. **C++ 中差异更极端**：C++ Lambda 比 C++ 普通慢 **150-750 倍**，远超 Python 的 10-20 倍

3.4 为什么 C++ 中这种差异更加明显？

背后主要有四个原因：

1. **编译器优化的极限**：C++ 的普通递归可以被编译器极度优化（内联、尾调用优化等），执行速度接近机器极限；而 `std::function` 的虚函数调用开销无法被完全优化掉
2. **类型系统的代价**：C++ 是静态类型语言，用 `std::function` 包装 lambda 会引入类型擦除（type erasure），每次调用都需要通过虚函数表分发；反过来说，Python 没有强制静态类型系统，很多调用要在运行时完成动态分发与类型处理，因此函数调用的基线开销本来就更大。
3. **Python 的"先天劣势"**：Python 的函数调用本身就慢（解释执行、动态分发），所以 lambda 和普通函数的相对差距较小；C++ 普通函数调用太快，lambda 的相对开销就被放大了
4. **内存布局**：C++ 的 `std::function` 需要在堆上分配闭包对象，而 Python 的函数对象本来就是堆分配的，相对差距不明显

结论：Lambda 演算理论上很美，但在实际工程中，性能代价是显著的。

不过，Lambda 演算依然有非常重要的优点：

1. **理论基础统一**：它提供了一个极简但完备的计算模型，让我们能用统一框架讨论"什么是可计算"。
2. **高可组合性**：函数作为一等公民，天然支持组合与抽象，很多现代函数式编程思想都源于此。
3. **形式化推理友好**：表达式结构简单，便于做等价变换、正确性证明与程序语义分析。
4. **对编译器与语言设计启发深远**：闭包、作用域、柯里化、惰性求值等核心机制，都可以在 Lambda 演算中找到清晰原型。

换句话说，Lambda 演算未必是工程里最快的实现方式，但它依然是理解编程语言本质与优化方向的一套重要"坐标系"。
